Foundations of agentic AI on AWS

AWS Prescriptive Guidance

Copyright © 2026 Amazon Web Services, Inc. and/or its affiliates. All rights reserved.

AWS Prescriptive Guidance: Foundations of agentic AI on AWS

Copyright © 2026 Amazon Web Services, Inc. and/or its affiliates. All rights reserved.

Amazon's trademarks and trade dress may not be used in connection with any product or service that is not Amazon's, in any manner that is likely to cause confusion among customers, or in any manner that disparages or discredits Amazon. All other trademarks not owned by Amazon are the property of their respective owners, who may or may not be affiliated with, connected to, or sponsored by Amazon.

Table of Contents

Foundations of agentic AI on AWS

Aaron Sempf, Tod Golding, Andrew Hooker, and Joshua Samuel, Amazon Web Services

In a world of increasingly intelligent, distributed, and autonomous systems, the concept of an *agent*—an entity that can perceive its environment, reason about its state, and act with intent—has become foundational. Agents are not merely programs that execute instructions; they are goal-oriented, context-aware entities that make decisions on behalf of users, systems, or organizations. Their emergence reflects a shift in how you build and think about software: a shift from procedural logic and reactive automation to systems that operate with autonomy and purpose.

At the intersection of AI, distributed systems, and software engineering lies a powerful paradigm known as *agentic AI*. This new generation of intelligent systems consists of software agents that are capable of adaptive behavior, complex coordination, and delegated decision-making.

This guide introduces the principles that define modern software agents and outlines their evolution toward agentic AI. To explain this shift, the guide provides the conceptual background and then traces the evolution of software agents to agentic AI:

- Introduction to software agents defines software agents, compares them to traditional software components, and introduces the essential characteristics that differentiate agentic behavior from traditional automation by drawing from established frameworks.
- The purpose of software agents examines why software agents exist, what roles they fulfill, what problems they solve, and how they enable intelligent delegation, reduce cognitive load, and support adaptive behavior in dynamic environments.
- The evolution of software agents traces the intellectual and technological milestones that shaped software agents, from early concepts of autonomy and concurrency to the emergence of multi-agent systems and formal agent architectures, resulting in the convergence with generative AI.
- Software agents to agentic AI introduces agentic AI as the culmination of decades of progress that combines distributed agent models with foundation models, serverless compute, and orchestration protocols. This section describes how this convergence enables a new generation of intelligent, tool-using agents that operate with autonomy, asynchronicity, and true agency at scale.

Intended audience

This guide is designed for architects, developers, and technology leaders who want to understand the history, main concepts, and evolution of software agents to agentic AI before they adopt this technology for modern cloud solutions on AWS.

Objectives

Adopting agentic architectures helps organizations:

- Accelerate time to value: Automate and scale knowledge work, and reduce manual effort and latency.
- Improve customer engagement: Deliver intelligent assistants across domains.
- Reduce operational costs: Automate decision flows that previously required human input or oversight.
- Drive innovation and differentiation: Build intelligent products that adapt, learn, and compete in real time.
- Modernize legacy workflows: Reframe scripts and monoliths into modular reasoning agents.

About this content series

This guide is part of a series about agentic AI on AWS. For more information and to view the other guides in this series, see Agentic AI on the AWS Prescriptive Guidance website.

Introduction to software agents

The concept of software agents has evolved significantly from its foundations in autonomous entities in the 1960s to its formal exploration in the early 1990s. As digital systems grow increasingly complex—from deterministic scripts to adaptive, intelligent applications—software agents have become essential building blocks for enabling autonomous, context-aware, and goal-driven behavior in computing systems. In the context of cloud-native and AI-enhanced architectures, particularly with the advent of generative AI, large language models (LLMs), and platforms such as Amazon Bedrock, software agents are being redefined through new lenses of capability and scale.

This introduction draws from the seminal work Software Agents: An Overview by Hyacinth S. Nwana (Nwana 1996). It defines software agents, discusses their conceptual roots, and extends the discussion into a contemporary framework to define three overarching principles of modern software agents: *autonomy*, *asynchronicity*, and *agency*. These principles distinguish software agents from other types of services or applications, and enable these agents to operate with purpose, resilience, and intelligence in distributed, real-time environments.

From autonomy to distributed intelligence

Before the term *software agent* entered the mainstream, early computing research explored the idea of *autonomous digital entities*, which are systems that are capable of acting independently, reacting to inputs, and making decisions based on internal rules or objectives. These early ideas laid the conceptual groundwork for what would become the agent paradigm. (For a historical timeline, see the section The evolution of software agents later in this guide.)

Early concepts of autonomy

The notion of machines or programs that act independently from human operators has intrigued system designers for decades. Early work in cybernetics, artificial intelligence, and control systems examined how software could exhibit self-regulating behavior, respond dynamically to changes, and operate without continuous human supervision.

These ideas introduced *autonomy* as a core attribute of intelligent systems and set the stage for the emergence of software that could *decide and act*, instead of only *reacting* or *executing*.

The actor model and asynchronous execution

In the 1970s, the *actor model*, which was introduced in the paper A Universal Modular ACTOR Formalism for Artificial Intelligence (Hewitt et al. 1973), provided a formal framework for thinking about decentralized, message-driven computation. In this model, actors are independent entities that communicate exclusively by passing asynchronous messages, and enable scalable, concurrent, and fault-tolerant systems.

The actor model emphasized three key attributes that continue to influence modern agent design:

- Isolation of state and behavior
- Asynchronous interaction between entities
- Dynamic creation and delegation of tasks

These attributes aligned with the needs of distributed systems and prefigured the operational characteristics of software agents in cloud-native environments.

Distributed intelligence and multi-agent systems

As computing systems became more interconnected after the 1960s, researchers explored distributed artificial intelligence (DAI). This field focused on how multiple autonomous entities could work collaboratively or competitively across a system. DAI led to the development of multi-agent systems, where each agent has local goals, perception, and reasoning but also operates within a broader, interconnected environment.

This vision of distributed intelligence, where decision-making is decentralized and emergent behavior arises from agent interaction, remains central to how modern agent-based systems are conceived and built.

Nwana's typology and the rise of software agents

The formalization of the software agent concept in the mid-1990s marked a turning point in the evolution of intelligent systems. Among the most influential contributions to this formalization is Hyacinth S. Nwana's seminal paper, Software Agents: An Overview (Nwana 1996), which provided one of the first comprehensive frameworks for categorizing and understanding software agents across various dimensions.

In this paper, Nwana surveys the state of software agent research and identifies a growing divergence in how agents were being defined and implemented. The paper highlights the need for a common conceptual framework and proposes a typology that classifies agents according to their key capabilities. It reviews representative agent systems from academia and industry, distinguishes agents from traditional programs and objects, and outlines the challenges and opportunities in agent-based computing.

Nwana emphasizes that software agents are not a monolithic concept but exist along a spectrum of sophistication and capability. The typology serves to clarify this landscape and guide future design and research.

Nwana defines a software agent as a software entity that functions continuously and autonomously in a particular environment, which is often inhabited by other agents and processes. This definition emphasizes two key characteristics:

- Continuity: The agent operates persistently over time, without requiring constant human intervention.
- Autonomy: The agent has the capability to make decisions and act on them independently, based on its perception of the environment.

This definition, combined with Nwana's agent typology, emphasizes delegated authority (through autonomy) and proactivity as foundational characteristics of agents. It differentiates between agents and subroutines or services by highlighting the agent's ability to act independently on behalf of another entity and to initiate behavior in pursuit of goals, instead of only responding to direct commands.

Nwana's agent typology

To further differentiate among various types of agents, Nwana introduces a classification system based on six key attributes:

- Autonomy: The agent operates without direct intervention from humans or others.
- Social ability: The agent interacts with other agents or humans by using communication mechanisms.
- Reactivity: The agent perceives its environment and responds in a timely manner.
- Proactivity: The agent exhibits goal-directed behavior by taking the initiative.
- Adaptability and learning: The agent improves its performance over time through experience.

- Mobility: The agent can move across different system environments or networks.

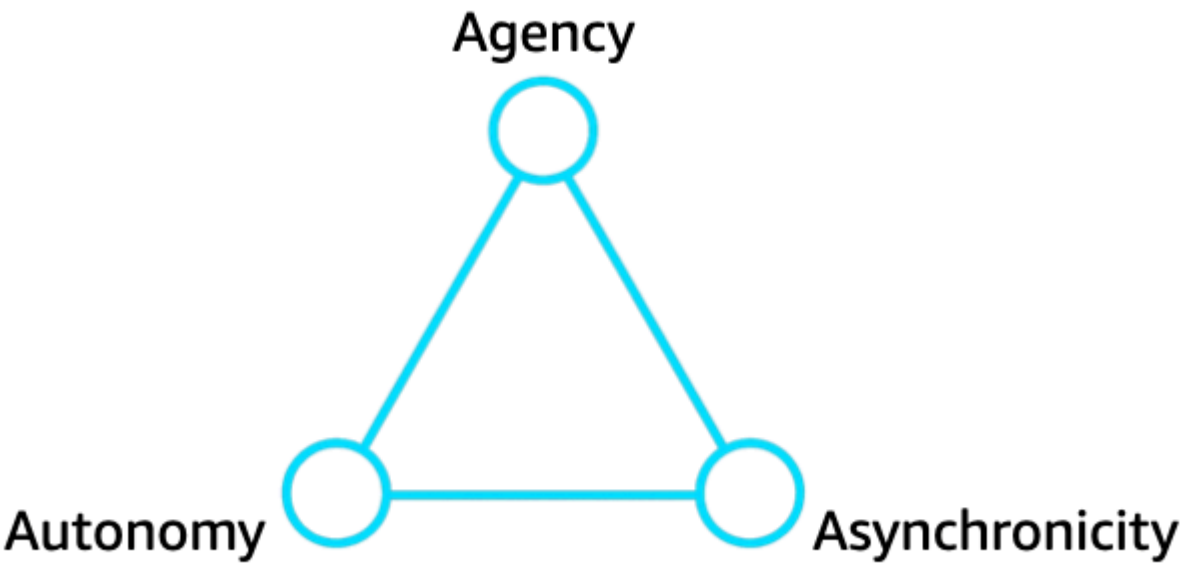
From typology to modern agentic principles

Nwana's work served as both a taxonomy and a foundational lens through which the computing community could evaluate the evolving forms of agency in software. His emphasis on autonomy, proactivity, and the concept of acting on behalf of a user or system laid the groundwork for what we now consider agentic behavior.

Although the technologies and environments have changed, especially with the rise of generative AI, serverless infrastructure, and multi-agent orchestration frameworks, the foundational insights from Nwana's work remain relevant. They provide a critical bridge between early agent theory and the three modern pillars of software agents.

The three pillars of modern software agents

In the context of today's AI-powered platforms, microservice architectures, and event-driven systems, software agents can be defined by three interdependent principles that distinguish them from standard services or automation scripts: autonomy, asynchronicity, and agency. In the following illustration and in subsequent diagrams, the triangle represents these three pillars of modern software agents.



Autonomy

Modern agents operate independently. They make decisions based on internal state and environmental context without requiring human prompts. This enables them to react to data in real-time, manage their own lifecycle, and adjust their behavior based on goals and situational inputs.

Autonomy is the foundation of agent behavior. It allows agents to function without continuous supervision or hardcoded control flows.

Asynchronicity

Agents are fundamentally asynchronous. This means that they respond to events, signals, and stimuli as they occur, without relying on blocking calls or linear workflows. This characteristic enables scalable, non-blocking communication, responsiveness in distributed environments, and loose coupling between components.

Through asynchronicity, agents can participate in real-time systems and coordinate with other services or agents fluidly and efficiently.

Agency as the defining principle

Autonomy and asynchronicity are necessary, but these features alone aren't sufficient to make a system a true software agent. The critical differentiator is agency, which introduces:

- Goal-directed behavior: Agents pursue objectives and evaluate progress toward them.
- Decision-making: Agents assess options and choose actions based on rules, models, or learned policies.
- Delegated intent: Agents act on behalf of a person, system, or organization and have an embedded sense of purpose.
- Contextual reasoning: Agents incorporate memory or models of their environment to guide behavior intelligently.

A system that is autonomous and asynchronous might still be a reactive service. What makes it a software agent is its ability to act with intention and purpose, to be *agentic*.

Agency with purpose

The principles of autonomy, asynchronicity, and agency enable systems to operate intelligently, adaptively, and independently across distributed environments. These principles are rooted in decades of conceptual and architectural evolution, and now underpin many of the most advanced AI systems being built today.

In this new era of generative AI, goal-oriented orchestration, and multi-agent collaboration, it's essential to understand what makes a software agent truly agentic. Recognizing agency as the defining characteristic helps us move beyond automation and into the realm of autonomous intelligence with purpose.

The purpose of software agents

As modern systems have become increasingly complex, distributed, and intelligent, the role of software agents has gained prominence across domains that range from autonomous operations to user-assistive technologies. But what is the underlying purpose of software agents? Why do we design systems that go beyond scripts, services, or static models, and instead delegate tasks to entities that are capable of perceiving, reasoning, and acting?

This section explores the fundamental purpose of software agents: to enable intelligent delegation of tasks within dynamic environments, with a focus on autonomy, adaptability, and purposeful action. It introduces the conceptual foundation of software agents, traces their cognitive structure, and outlines the real-world problems that they are uniquely equipped to solve.

From the actor model to agent cognition

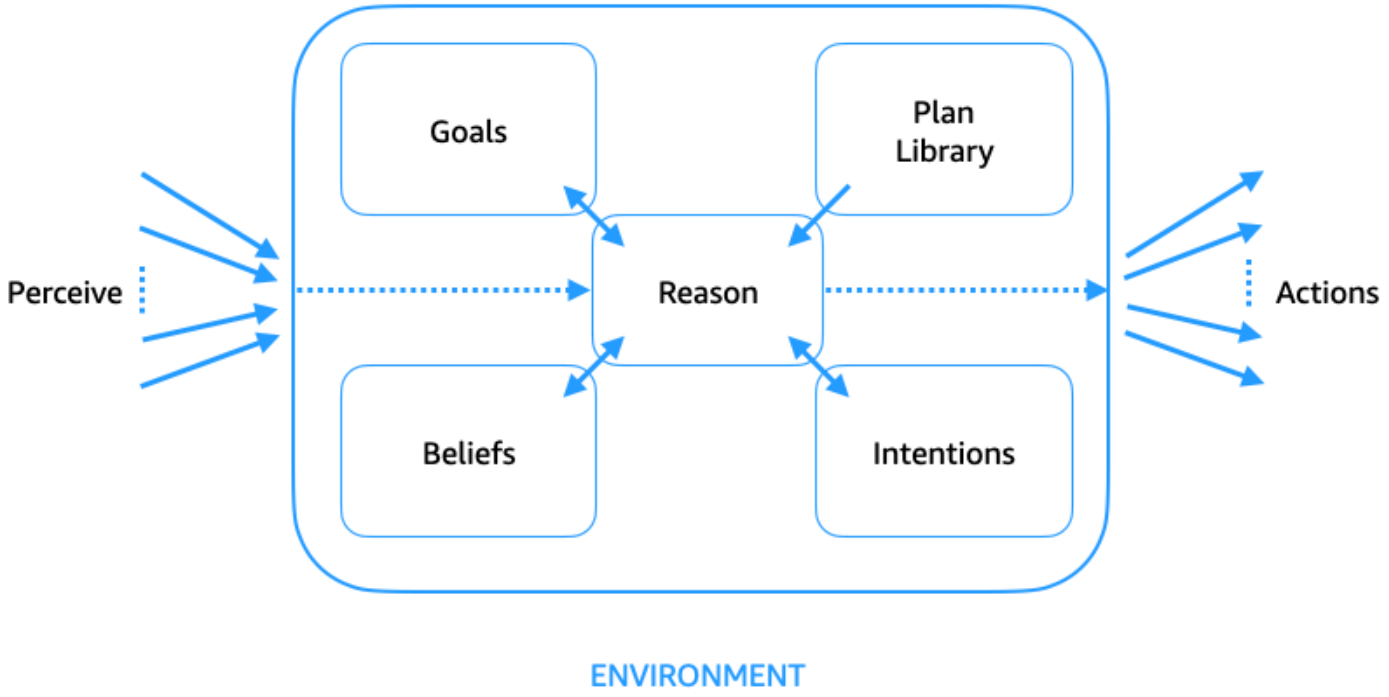
The purpose and structure of software agents are grounded in ideas that emerged from early computation models, particularly the actor model that was introduced by Carl Hewitt in the 1970s (Hewitt et al. 1973).

The actor model treats computation as a collection of independent, concurrently executing entities called *actors*. Each actor encapsulates its own state, interacts solely through asynchronous message passing, and can create new actors and delegate tasks.

This model provided the conceptual foundation for decentralized reasoning, reactivity, and isolation—all of which underpin the behavioral architecture of modern software agents.

The agent function: perceive, reason, act

At the core of every software agent is a cognitive cycle that is often described as the *perceive, reason, act* loop. This process is illustrated in the following diagram. It defines how agents operate autonomously in dynamic environments.



- **Perceive**: Agents gather information (for example, events, sensor inputs, or API signals) from the environment and update their internal state or beliefs.
- **Reason**: Agents analyze current beliefs, goals, and contextual knowledge by using a plan library or logic system. This process might involve goal prioritization, conflict resolution, or intention selection.
- **Act**: Agents select and execute actions that move them closer to achieving their delegated goals.

This architecture supports the ability of agents to function beyond rigid programming and enables flexible, context-sensitive, and goal-directed behavior. It forms the mental framework that guides the broader purposes of software agents.

Autonomous collaboration and intentionality

The purpose of software agents is to bring autonomy, context-awareness, and intelligent delegation to modern computing. Because agents are built on the principles of the actor model and embodied in the perceive, reason, act cycle, they enable systems that are not only reactive, but proactive and purposeful.

Agents empower software to decide, adapt, and act in complex environments. They represent users, interpret goals, and implement tasks at machine speed. As we move deeper into the era of agentic AI, software agents are becoming the operational interface between human intent and intelligent digital action.

Delegating intent

Unlike traditional software components, software agents exist to act on behalf of something else: a user, another system, or a higher-level service. They carry *delegated intent*, which means that they:

- Operate independently after initiation.
- Make choices that are aligned with the goals of the delegator.
- Navigate uncertainty and trade-offs in execution.

Agents bridge the gap between *instructions* and *outcomes*, which allows users to express intent at a higher level of abstraction instead of requiring explicit instructions.

Operating in dynamic, unpredictable environments

Software agents are designed for environments where conditions change constantly, data arrives in real time, and control and context are distributed.

Unlike static programs that require exact inputs or synchronous execution, agents adapt to their surroundings and respond dynamically. This is a vital capability in cloud-native infrastructure, edge computing, Internet of Things (IoT) networks, and real-time decision-making systems.

Reducing human cognitive load

One of the primary purposes of software agents is to reduce the cognitive and operational burden on humans. Agents can:

- Continuously monitor systems and workflows.
- Detect and respond to predefined or emergent conditions.
- Automate repetitive, high-volume decisions.
- React to environmental changes with minimal latency.

When decision-making shifts from users to agents, systems become more responsive, resilient, and human-centric, and can adapt in real time to new information or disruptions. This enables faster

reaction turnaround as well as greater operational continuity in high-complexity or high-scale environments. The result is a shift in human focus, from micro-level decision-making to strategic oversight and creative problem-solving.

Enabling distributed intelligence

The ability of software agents to operate individually or collectively enables the design of multi-agent systems (MAS) that coordinate across environments or organizations. These systems can distribute tasks intelligently and negotiate, cooperate, or compete toward composite goals.

For example, in a global supply chain system, individual agents manage factories, shipping, warehouses, and last-mile delivery. Each agent operates with local autonomy: Factory agents optimize production based on resource constraints, warehouse agents adjust inventory flows in real time, and delivery agents reroute shipments based on traffic and customer availability.

These agents communicate and coordinate dynamically, and adapt to disruptions such as port delays or truck failures without centralized control. The system's overall intelligence emerges from these interactions and enables resilient, optimized logistics that are beyond the capabilities of a single component.

In this model, agents act as nodes in a broader intelligence fabric. They form emergent systems that are capable of solving problems that no single component could handle alone.

Acting with purpose, not only reaction

Automation alone is insufficient in complex systems. The defining purpose of a software agent is to act with purpose and to evaluate goals, weigh context, and make informed choices. This means that software agents pursue goals instead of only responding to triggers. They can revise beliefs and intentions based on experience or feedback. In this context, beliefs refer to the agent's internal representation of the environment (for example, "package X is in warehouse A"), based on its perceptions (input and sensors). Intentions refer to the plans that the agent chooses to achieve a goal (for example, "use delivery route B and notify the recipient"). Agents can also escalate, defer, or adapt actions as necessary.

This intentionality is what makes software agents not just reactive executors, but autonomous collaborators in intelligent systems.

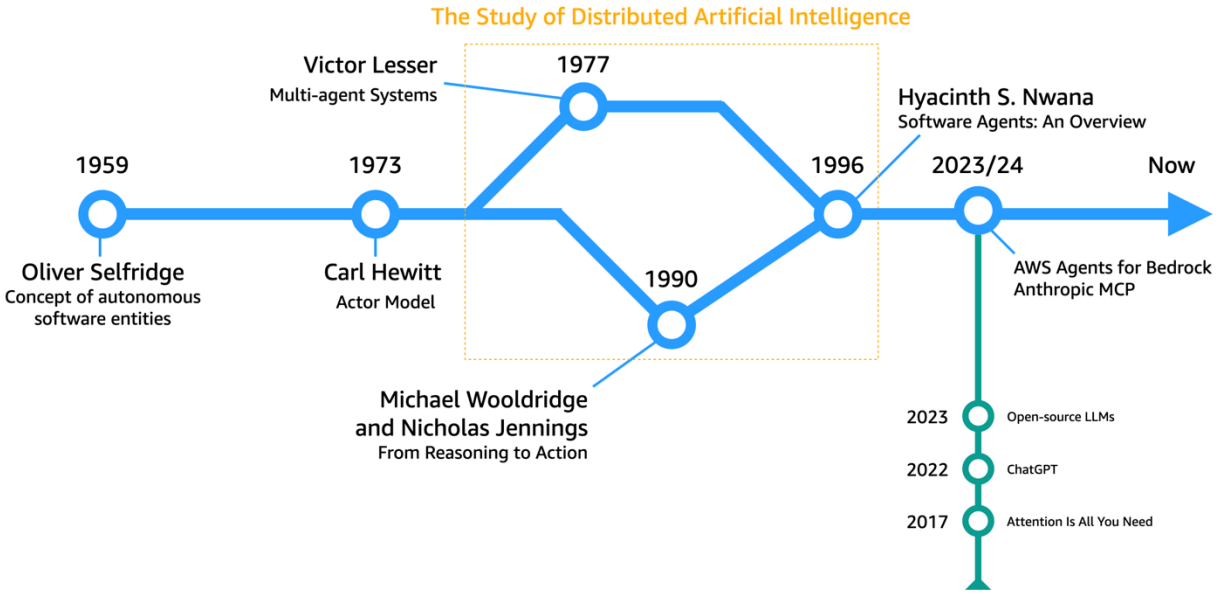
The evolution of software agents

The journey from simple automated systems to intelligent, autonomous, and goal-directed software agents reflects decades of evolution in computer science, artificial intelligence, and distributed systems.

This evolution was followed by the rise of machine learning, which shifted the paradigm from handcrafted rules to statistical pattern recognition. These systems could learn from data and enabled advances in perception, classification, and decision-making.

Large language models (LLMs) represent a convergence of scale, architecture, and unsupervised learning. LLMs can reason, generate, and adapt tasks with little or no task-specific training. By combining LLMs with scalable cloud-native infrastructure and composable architectures, we are now achieving the full vision of agentic AI: intelligent software agents that can operate with autonomy, context-awareness, and adaptability at enterprise scale.

This section explores the history of software agents from foundational theory to modern practice, as illustrated in the following diagram. It highlights the convergence of distributed artificial intelligence (DAI) and transformer-based generative AI, and identifies the key milestones that have shaped the emergence of agentic AI.



Foundations of software agents

1959 - Oliver Selfridge: the birth of autonomy in software

The roots of software agents trace back to Oliver Selfridge, who introduced the concept of *autonomous software entities (demons)*—programs that are capable of perceiving their environment and acting independently (Selfridge 1959). His early work in machine perception and learning laid the philosophical groundwork for future notions of agents as independent, intelligent systems.

1973 - Carl Hewitt: the actor model

A pivotal advancement came with Carl Hewitt's actor model (Hewitt et al. 1973), which is a formal computational model that describes agents as independent, concurrent entities. In this model, agents can encapsulate their own state and behavior, communicate by using asynchronous message passing, and dynamically create other actors and delegate tasks to them.

The actor model provided both the theoretical foundation and the architectural paradigm for distributed, agent-based systems. This model prefigured modern concurrency implementations such as the Erlang programming language and the Akka framework.

Maturing the field: from reasoning to action

1977 - Victor Lesser: multi-agent systems

In the late 1970s, distributed artificial intelligence (DAI) emerged. It was championed by Victor Lesser, who is widely recognized for pioneering multi-agent systems (MAS). His work focused on how independent software entities could cooperate, coordinate, and negotiate (see the Resources section). This development led to systems that were capable of solving complex problems collectively—an essential leap in building distributed intelligence.

1990s - Michael Wooldridge and Nicholas Jennings: the agent spectrum

By the 1990s, the distributed intelligence field had matured with contributions from researchers such as Michael Wooldridge and Nicholas Jennings. These scholars categorized agents along a spectrum, from reactive to deliberative, from non-cognitive systems to goal-driven, reasoning agents (Wooldridge and Jennings 1995). Their work emphasized that agents were no longer

abstract ideas but were being applied across a wide range of practical domains, from robotics to enterprise software.

These researchers also introduced a shift in focus: from centralized reasoning to distributed action. Agents were no longer just thinkers—they were doers that operated in real-time environments with autonomy and purpose.

1996 - Hyacinth S. Nwana: formalizing the agent concept

In 1996, Hyacinth S. Nwana published the influential paper Software Agents: An Overview, which provided the most comprehensive classification of agents to date. His typology included attributes such as autonomy, social ability, reactivity, proactivity, learning, and mobility, and differentiated between software agents and traditional software constructs.

Nwana also offered a now widely accepted definition, paraphrased: *A software agent is a software-based computer program that acts for a user or other program in a relationship of agency, which derives from the notion of delegation.*

This formalization was instrumental in transitioning software agents from theoretical constructs to real-world applications. It gave rise to a generation of agent-based systems across fields such as telecommunications, workflow automation, and intelligent assistants.

Nwana's work sits at the convergence point of early distributed AI research and the operational architectures of modern agents. It is a crucial bridge between the cognitive theory of agents and their practical deployment in today's systems.

A parallel timeline: the rise of large language models

While agent frameworks evolved, a parallel and convergent revolution was happening in natural language processing and machine learning:

- **2017 – transformers**: The paper Attention Is All You Need (Vaswani et al. 2017) introduced the transformer architecture, which dramatically improved how machines process and generate language.
- **2022 – ChatGPT**: OpenAI released a chat-based interface to GPT-3.5 called ChatGPT, which enabled natural, interactive conversation with a general-purpose AI system.
- **2023 – open source LLMs**: The releases of Llama, Falcon, and Mistral made powerful models widely accessible and accelerated the development of agent frameworks in open source and enterprise environments.

These innovations turned language models into reasoning engines that are capable of parsing context, planning actions, and chaining responses, and LLMs became key enablers of intelligent software agents.

Timelines converge: the emergence of agentic AI

2023-2024 - enterprise-grade agent platforms

The convergence of distributed software agent architectures and transformer-based LLMs culminated in the rise of agentic AI.

- Amazon Bedrock Agents introduced a fully managed way to build goal-driven, tool-using software agents by using foundation models from Amazon Bedrock.
- The Model Context Protocol (MCP) from Anthropic defined a method for large language models to access, and interact with, external tools, environments, and memory. This is key for contextual, persistent, and autonomous behavior.

These two milestones represent the synthesis of agency and intelligence. Agents were no longer limited to static workflows or rigid automation. They could now reason across multiple steps, coordinate with tools and APIs, maintain contextual state, and learn and adapt over time.

January-June 2025 - expanded enterprise capabilities

In the first half of 2025, the agentic AI landscape expanded significantly with new enterprise capabilities. In February 2025, Anthropic released Claude 3.7 Sonnet, which was the first hybrid reasoning model on the market, and the MCP specification gained widespread adoption.

AI coding assistants such as Amazon Q Developer, Cursor, and WindSurf integrated MCP to standardize code generation, repository analysis, and development workflows. The MCP March 2025 release introduced significant enterprise-ready features, including OAuth 2.1 security integration, expanded resource types for diverse data access, and enhanced connectivity options through Streamable HTTP. Building on this foundation, AWS announced in May 2025 that it was joining the MCP steering committee and contributing to new agent-to-agent communication capabilities. This further strengthens the protocol's position as an industry standard for agentic AI interoperability.

In May 2025, AWS strengthened customer options for building agentic AI workflows by open sourcing the Strands Agents framework. This provider-independent and model-agnostic framework

enables developers to use foundation models across platforms while maintaining deep AWS service integration. As highlighted in the AWS Open Source Blog, Strands Agents follows a model-first design philosophy that places foundation models at the core of agent intelligence. This makes it easier for customers to build and deploy sophisticated AI agents for their specific use cases.

Emergence - agentic AI

The evolution of software agents, from early ideas of autonomy to modern, LLM-enabled orchestration, has been long and layered. What began with Oliver Selfridge's vision of perceiving programs has grown into a robust ecosystem of intelligent, context-aware, goal-driven software agents that can collaborate, adapt, and reason.

The convergence of distributed artificial intelligence (DAI) and transformer-based generative AI marks the beginning of a new era in which software agents are no longer only tools, but autonomous actors in intelligent systems.

Agentic AI represents the next evolution in software systems. It provides a class of intelligent agents that are autonomous, asynchronous, and agentic, and can act with delegated intent and operate purposefully within dynamic, distributed environments. Agentic AI unifies the following:

- The architectural lineage of multi-agent systems and the actor model
- The cognitive model of perceive, reason, act
- The generative power of LLMs and transformers
- The operational flexibility of cloud-native and serverless computing

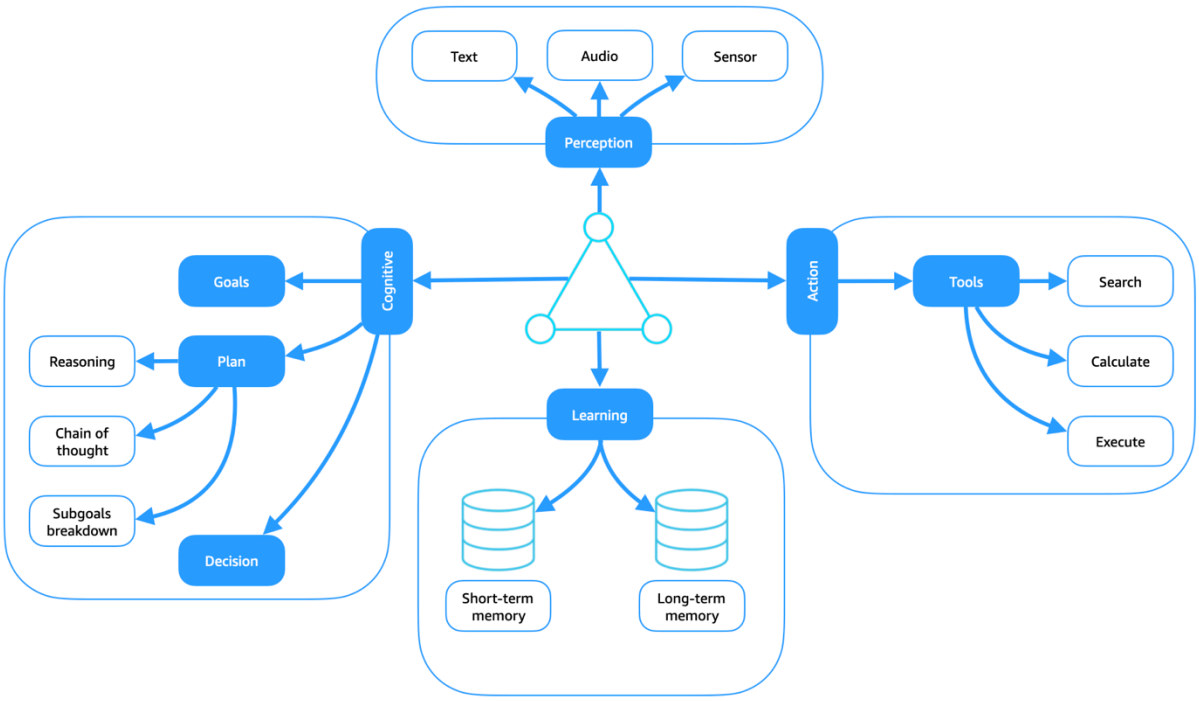
Software agents to agentic AI

Software agents are autonomous digital entities that are designed to perceive their environment, reason about their goals, and act accordingly. Unlike traditional software programs that follow fixed logic, agents adapt their behavior based on contextual inputs and decision frameworks. This makes them ideal for dynamic, distributed environments such as cloud-native systems, robotics, intelligent automation, and now, generative AI orchestration.

This section introduces the core building blocks of software agents and explains how these components interact within traditional architectures based on the perceive, reason, act model. It discusses how generative AI, particularly large language models (LLMs), has transformed the way software agents reason and plan. This marks a fundamental shift from rule-based systems to the data-driven, learned intelligence of agentic AI.

Core building blocks of software agents

The following diagram presents the key functional modules found in most intelligent agents. Each component contributes to the agent's ability to operate autonomously in complex environments.



In the context of the perceive, reason, act loop, an agent's reasoning capability is distributed across both its cognitive and learning modules. Through the integration of memory and learning,

the agent develops adaptive reasoning grounded in past experience. As the agent acts within its environment, it creates an emergent feedback loop: Each action influences future perceptions, and the resulting experience is incorporated into memory and internal models through the learning module. This continuous loop of perception, reasoning, and action enables the agent to improve over time and completes the full perceive, reason, act cycle.

Perception module

The perception module enables the agent to interface with its environment through diverse input modalities such as text, audio, and sensors. These inputs form the raw data that all reasoning and action are based on. Text inputs might include natural language prompts, structured commands, or documents. Audio inputs encompass spoken instructions or environmental sounds. Sensor inputs include physical data such as visual feeds, motion signals, or GPS coordinates. The core function of perception is to extract meaningful features and representations from this raw data. This allows the agent to construct an accurate and actionable understanding of its current context. The process might involve feature extraction, object or event recognition, and semantic interpretation, and forms the critical first step in the perceive, reason, act loop. Effective perception ensures that downstream reasoning and decision-making are grounded in relevant, up-to-date situational awareness.

Cognitive module

The cognitive module serves as the deliberative core of the software agent. It is responsible for interpreting perceptions, forming intent, and guiding purposeful behavior through goal-driven planning and decision-making. This module transforms inputs into structured reasoning processes, which enables the agent to operate intentionally rather than reactively. These processes are managed through three key submodules: goals, planning, and decision-making.

Goals submodule

The goals submodule defines the agent's intent and direction. Goals can be explicit (for example, "navigate to a location" or "submit a report") or implicit (for example, "maximize user engagement" or "minimize latency"). They are central to the agent's reasoning cycle, and provide a target state for its planning and decisions.

The agent continuously evaluates progress toward its goals and might reprioritize or regenerate goals based on new perceptions or learning. This goal awareness keeps the agent adaptable in dynamic environments.

Planning submodule

The planning submodule constructs strategies to achieve the agent's current goals. It generates action sequences, decomposes tasks hierarchically, and selects from predefined or dynamically generated plans.

To operate effectively in non-deterministic or changing environments, planning is not static. Modern agents can generate chain-of-thought sequences, introduce subgoals as intermediate steps, and revise plans in real time when conditions shift.

This submodule connects closely with memory and learning, and allows the agent to refine its planning over time based on past outcomes.

Decision-making submodule

The decision-making submodule evaluates available plans and actions to select the most appropriate next step. It integrates input from perception, the current plan, the agent's goals, and environmental context.

Decision-making accounts for:

- Trade-offs between conflicting goals
- Confidence thresholds (for example, uncertainty in perception)
- Consequences of actions
- The agent's learned experience

Depending on the architecture, agents might rely on symbolic reasoning, heuristics, reinforcement learning, or language models (LLMs) to make informed decisions. This process keeps the agent's behavior context-aware, goal-aligned, and adaptive.

Action module

The action module is responsible for executing the agent's selected decisions and interfacing with the external world or internal systems to produce meaningful effects. It represents the Act phase of the perceive, reason, act loop, where intent is transformed into behavior.

When the cognitive module selects an action, the action module coordinates execution through specialized submodules, where each submodule aligns with the agent's integrated environment:

- Physical actuation: For agents that are embedded in robotic systems or IoT devices, this submodule translates decisions into real-world physical movements or hardware-level instructions.

  Examples: steering a robot, triggering a valve, turning on a sensor.

- Integrated interaction: This submodule handles non-physical but externally visible actions such as interacting with software systems, platforms, or APIs.

  Examples: sending a command to a cloud service, updating a database, submitting a report by calling an API.

- Tool invocation: Agents often extend their capabilities by using specialized tools to accomplish sub-tasks such as the following:
  - Search: querying structured or unstructured knowledge sources
  - Summarization: compressing large text inputs into high-level overviews
  - Calculation: performing logical, numerical, or symbolic computation

  Tool invocation enables complex behavior composition through modular, callable skills.

Learning module

The learning module enables agents to adapt, generalize, and improve over time based on experience. It supports the reasoning process by continuously refining the agent's internal models, strategies, and decision policies by using feedback from perception and action.

This module operates in coordination with both short-term and long-term memory:

- Short-term memory: Stores transient context, such as dialogue state, current task information, and recent observations. It helps the agent maintain continuity within interactions and tasks.
- Long-term memory: Encodes persistent knowledge from past experiences, including previously encountered goals, outcomes of actions, and environmental states. Long-term memory enables the agent to recognize patterns, reuse strategies, and avoid repeating mistakes.

Learning modes

The learning module supports a range of paradigms, such as supervised, unsupervised, and reinforcement learning, which support different environments and agent roles:

- Supervised learning: Updates internal models based on labeled examples, often from human feedback or training datasets.

  Example: learning to classify user intent based on previous conversations.
- Unsupervised learning: Identifies hidden patterns or structures in data without explicit labels.

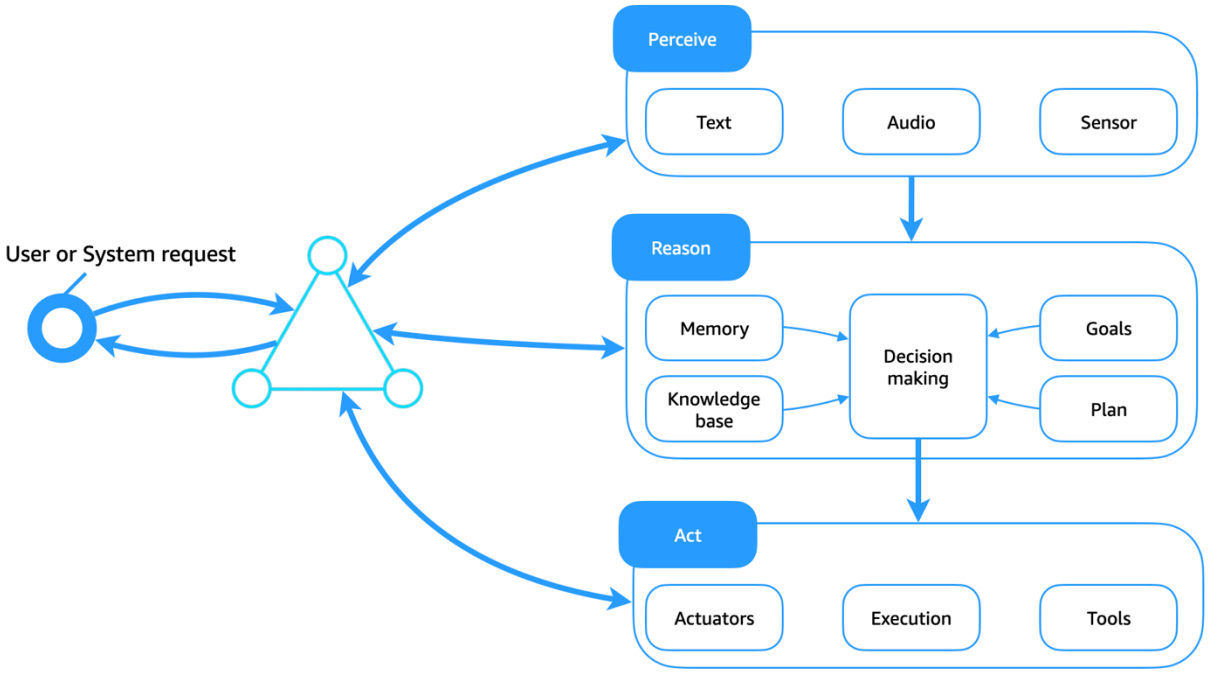
  Example: clustering environmental signals to detect anomalies.
- Reinforcement learning: Optimizes behavior through trial and error by maximizing cumulative reward in interactive environments.

  Example: learning which strategy leads to the fastest task completion.

Learning integrates tightly with the agent's cognitive module. It refines planning strategies based on past outcomes, enhances decision-making through the evaluation of historical success, and continuously improves the mapping between perception and action. Through this closed learning and feedback loop, agents evolve beyond reactive execution to become self-improving systems that are capable of adapting to new goals, conditions, and contexts over time.

Traditional agent architecture: perceive, reason, act

The following diagram illustrates how the building blocks discussed in the previous section operate under the perceive, reason, act cycle.



Perceive module

The perceive module acts as the agent's sensory interface with the external world. It transforms raw environmental input into structured representations that inform reasoning. This includes handling multimodal data such as text, audio, or sensor signals.

- Text input may come from user commands, documents, or dialogue.
- Audio input includes spoken instructions or environmental sounds.
- Sensor input captures real-world signals such as motion, visual feeds, or GPS.

When the raw input has been ingested, the perception process performs feature extraction, followed by object or event recognition and semantic interpretation to create a meaningful model of the current situation. These outputs provide structured context for downstream decision-making and anchor the agent's reasoning in real-world observations.

Reason module

The reason module is the cognitive core of the agent. It evaluates context, formulates intent, and determines appropriate actions. This module orchestrates goal-driven behavior by using both learned knowledge and reasoning.

The reason module consists of tightly integrated submodules:

- Memory: Maintains dialogue state, task context, and episodic history in both short-term and long-term formats.
- Knowledge base: Provides access to symbolic rules, ontologies, or learned models (such as embeddings, facts, and policies).
- Goals and plans: Defines desired outcomes and constructs action strategies to achieve them. Goals can be dynamically updated and plans can be adaptively modified based on feedback.
- Decision-making: Acts as the central arbitration engine by weighing options, evaluating trade-offs, and selecting the next action. This submodule factors in confidence thresholds, goal alignment, and contextual constraints.

Together, these components allow the agent to reason about its environment, update beliefs, select paths, and behave in a coherent, adaptive manner. The reason module closes the gap between perception and behavior.

Act module

The act module executes the agent's selected decision by interfacing with either the digital or the physical environment to carry out tasks. This is where intention becomes action.

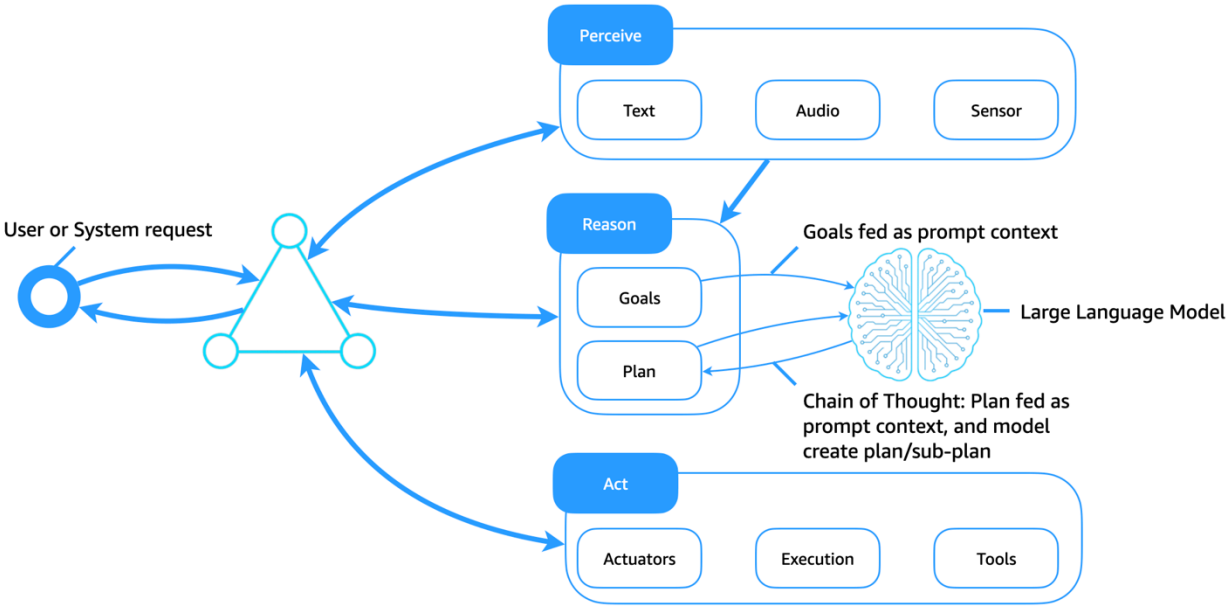
This module includes three functional channels:

- Actuators: For agents that have a physical presence (such as robots and IoT devices), controls hardware-level interactions such as movement, manipulation, or signaling.
- Execution: Handles software-based actions, including invoking APIs, dispatching commands, and updating systems.
- Tools: Enables functional capabilities such as search, summarization, code execution, calculation, and document handling. These tools are often dynamic and context-aware, which extends the agent's utility.

The outputs of the act module feed back into the environment and close the loop. These outcomes are perceived by the agent again. They update the agent's internal state and inform future decisions, thus completing the perceive, reason, act cycle.

Generative AI agents: replacing symbolic logic with LLMs

The following diagram illustrates how large language models (LLMs) now serve as a flexible and intelligent cognitive core for software agents. In contrast to traditional symbolic logic systems, which rely on static plan libraries and hand-coded rules, LLMs enable adaptive reasoning, contextual planning, and dynamic tool use, which transform how agents perceive, reason, and act.



Key enhancements

This architecture enhances the traditional agent architecture as follows:

- LLMs as cognitive engines: Goals, plans, and queries are passed into the model as prompt context. The LLM generates reasoning paths (such as chain of thought), decomposes tasks into sub-goals, and decides on next actions.
- Tool use through prompting: LLMs can be directed through tool use agents or reasoning and acting (ReAct) prompting to call APIs and to search, query, calculate, and interpret outputs.
- Context-aware planning: Agents generate or revise plans dynamically based on the agent's current goal, input environment, and feedback, without requiring hardcoded plan libraries.
- Prompt context as memory: Instead of using symbolic knowledge bases, agents encode memory, plans, and goals as prompt tokens that are passed to the model.
- Learning through few-shot, in-context learning: LLMs adapt behaviors through prompt engineering, which reduces the need for explicit retraining or rigid plan libraries.

Achieving long-term memory in LLM-based agents

Unlike traditional agents, which stored long-term memory in structured knowledge bases, generative AI agents must work within the context window limitations of LLMs. To extend memory and support persistent intelligence, generative AI agents use several complementary techniques:

agent store, Retrieval-Augmented Generation (RAG), in-context learning and prompt chaining, and pretraining.

Agent store: external long-term memory

Agent state, user history, decisions, and outcomes are stored in a long-term agent memory store (such as a vector database, object store, or document store). Relevant memories are retrieved on demand and injected into the LLM prompt context at runtime. This creates a persistent memory loop, where the agent retains continuity across sessions, tasks, or interactions.

RAG

RAG enhances LLM performance by combining retrieved knowledge with generative capabilities. When a goal or query is issued, the agent searches a retrieval index (for example, through a semantic search of documents, earlier conversations, or structured knowledge). The retrieved results are appended to the LLM prompt, which grounds the generation in external facts or personalized context. This method extends the agent's effective memory and improves reliability and factual correctness.

In-context learning and prompt chaining

Agents maintain short-term memory by using in-session token context and structured prompt chaining. Contextual elements, such as the current plan, previous action outcomes, and agent status, are passed between calls to guide behavior.

Continued pretraining and fine-tuning

For domain-specific agents, LLMs can be continued pretrained on custom collections such as logs, enterprise data, or product documentation. Alternatively, instruction fine-tuning or reinforcement learning from human feedback (RLHF) can embed agent-like behavior directly into the model. This shifts reasoning patterns from prompt-time logic into the model's internal representation, reduces prompt length, and improves efficiency.

Combined benefits in agentic AI

These techniques, when they're used together, enable generative AI agents to:

- Maintain contextual awareness over time.
- Adapt behavior based on user history or preferences.

- Make decisions by using up-to-date, factual, or private knowledge.
- Scale to enterprise use cases with persistent, compliant, and explainable behaviors.

By augmenting LLMs with external memory, retrieval layers, and continued training, agents can achieve a level of cognitive continuity and purpose that couldn't be achieved previously through symbolic systems alone.

Comparing traditional AI to software agents and agentic AI

The following table provides a detailed comparison of traditional AI, software agents, and agentic AI.

Characteristic	Traditional AI	Software agents	Agentic AI
Examples	Spam filters, image classifiers, recommendation engines	Chatbots, task schedulers, monitoring agents	AI assistants, autonomous developer agents, multi-agent LLM orchestrations
Execution model	Batch or synchronous	Event-driven or scheduled	Asynchronous, event-driven, and goal-driven
Autonomy	Limited; often requires human or external orchestration	Medium; operates independently within predefined bounds	High; acts independently with adaptive strategies
Reactivity	Reactive to input data	Reactive to environment and events	Reactive and proactive; anticipates and initiates actions
Proactivity	Rare	Present in some systems	Core attribute; drives goal-directed behavior

Communication	Minimal; usually standalone or API-bound	Inter-agent or agent-human messaging	Rich multi-agent and human-in-the-loop interaction
Decision-making	Model inference only (classification, prediction, and so on)	Symbolic reasoning , or rule-based or scripted decisions	Contextual, goal-based, dynamic reasoning (often LLM-enhanced)
Delegated intent	No; performs tasks defined directly by user	Partial; acts on behalf of users or systems that have limited scope	Yes; acts with delegated goals, often across services, users, or systems
Learning and adaptation	Often model-centric (for example., ML training)	Sometimes adaptive	Embedded learning, memory, or reasoning (for example, feedback, self-correction)
Agency	None; tools for humans	Implicit or basic	Explicit; operates with purpose, goals, and self-direction
Context awareness	Low; stateless or snapshot-based	Moderate; some state tracking	High; uses memory, situational context, and environment models
Infrastructure role	Embedded in apps or analytics pipelines	Middleware or service layer component	Composable agent mesh integrated with cloud, serverless, or edge systems

In summary:

- Traditional AI is tool-centric and functionally narrow. It focuses on prediction or classification.
- Traditional software agents introduce autonomy and basic communication, but they are often rule-bound or static.
- Agentic AI brings together autonomy, asynchrony, and agency. It enables intelligent, goal-driven entities that can reason, act, and adapt within complex systems. This makes agentic AI ideal for the cloud-native, AI-driven future.

Next steps

This guide discussed the history and foundations of agentic AI, which represents the evolution of traditional software agents into autonomous, intelligent systems that are powered by generative AI. It described how early software agents followed predefined rules and logic to automate tasks within fixed boundaries, and explained how agentic AI builds on this foundation by incorporating large language models, which enable agents to reason, learn, and adapt dynamically in open-ended environments.

You can explore agentic AI in depth by reviewing the following publications in this series:

- Operationalizing agentic AI on AWS provides an organizational strategy to transform agentic AI from isolated experiments into enterprise-scale, value-generating infrastructure.
- Agentic AI patterns and workflows on AWS discusses the foundational blueprints and modular constructs used to design, compose, and orchestrate goal-oriented AI agents.
- Agentic AI frameworks, protocols, and tools on AWS covers the software foundations, toolkits, and protocols to consider when you build your agentic AI solutions.
- Building serverless architectures for agentic AI on AWS discusses serverless systems as a natural foundation of modern AI workloads and describes how you can build AI-native serverless architectures in the AWS Cloud.
- Building multi-tenant architectures for agentic AI on AWS describes the use of AI agents in multi-tenant settings, including hosting considerations, deployment models, and control planes.

Resources

For more information about the concepts discussed in this guide, see the following guides and articles.

AWS references

- Amazon Bedrock Agents
- Amazon Q Developer
- Strands Agents SDK

Other references

- Hewitt, Carl, Peter Bishop, and Richard Steiger. "A Universal Modular ACTOR Formalism for Artificial Intelligence." *Proceedings of the 3rd International Joint Conference on Artificial Intelligence* (1973): 235-245. https://www.ijcai.org/Proceedings/73/Papers/027B.pdf
- Lesser, Victor R., relevant publications (see full list):
  - Lesser, Victor R. and Daniel D. Corkill. "Functionally Accurate, Cooperative Distributed Systems." *IEEE Transactions on Systems, Man, and Cybernetics* 11, no. 1 (1981): 81-96. https://ieeexplore.ieee.org/abstract/document/4308581
  - Decker, Keith S. and Victor R. Lesser. "Communication in the Service of Coordination." *AAAI Workshop on Planning for Interagent Communication* (1994). https://www.researchgate.net/profile/Victor-Lesser/publication/2768884_Communication_in_the_Service_of_Coordination/links/00b7d51cc2a0750cb4000000/Communication-in-the-Service-of-Coordination.pdf
  - Durfee, Edmund H., Victor R. Lesser, and Daniel D. Corkill. "Trends in Cooperative Distributed Problem Solving." *IEEE Transactions on knowledge and data Engineering* (1989). http://mas.cs.umass.edu/Documents/ieee-tkde89.pdf
  - Durfee, Edmund H., V.R. Lesser, and D.D. Corkill, "Distributed Artificial Intelligence." *Cooperation Through Communication in a Distributed Problem Solving-Network* (1987): 29-58. https://www.academia.edu/download/79885643/durf94_1.pdf
  - Lâasri, Brigitte, Hassan Lâasri, Susan Lander, and Victor Lesser. "A Generic Model for Intelligent Negotiating Agents." *International Journal of Cooperative Information Systems* 01, no. 02 (1992): 291-317. https://doi.org/10.1142/S0218215792000210

  - Lander, Susan E. and Victor R. Lesser. "Understanding the Role of Negotiation in Distributed Search Among Heterogeneous Agents." *IJCAI'93: Proceedings of the 13th international joint conference on Artifical intelligence* (1993): 438-444. https://www.ijcai.org/Proceedings/93-1/Papers/062.pdf
  - Lander, Susan, Victor R. Lesser, and Margaret E. Connell. "Conflict Resolution Strategies for Cooperating Expert Agents" *CKBS'90: Proceedings of the International Working Conference on Cooperating Knowledge Based Systems* (October 1990): 183-200. https://doi.org/10.1007/978-1-4471-1831-2_10
  - Prasad, M.V. Nagendra, Victor Lesser, and Susan E. Lander. "Learning Experiments in a Heterogeneous Multi-agent System." *IJCAI-95 Workshop on Adaptation and Learning in Multi-agent Systems* (1995): 59-64. https://www.researchgate.net/publication/2784280_Learning_Experiments_in_a_Heterogeneous_Multi-agent_System
- Nwana, Hyacinth S. "Software Agents: An Overview." *Knowledge Engineering Review* 11, no. 3 (October/November 1996): 205-244. https://teaching.shu.ac.uk/aces/rh1/elearning/multiagents/introduction/nwana.pdf
- Selfridge, Oliver G. "Pandemonium: A Paradigm for Learning." *Mechanization of Thought Processes: Proceedings of a Symposium Held at the National Physical Laboratory* 1 (1959): 511–529. https://aitopics.org/download/classics:504E1BAC
- Vaswani, Ashish, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. "Attention Is All You Need." *Proceedings of the 31st Conference on Neural Information Processing Systems (NIPS)*. Advances in Neural Information Processing Systems 30 (2017): 5998-6008. https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf
- Wooldridge, Michael and Nicholas R. Jennings. "Intelligent Agents: Theory and Practice." *Knowledge Engineering Review* 10, no. 2 (January 1995): 115-152. https://www.cs.cmu.edu/~motionplanning/papers/sbp_papers/integrated1/woodridge_intelligent_agents.pdf

Document history

The following table describes significant changes to this guide.

Change	Description	Date
Initial publication	—	July 14, 2025